



**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Дальневосточный федеральный университет»
(ДВФУ)**

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Кафедра математического и компьютерного моделирования

Избосарова Жасмина Олимжоновна

ЛАБОРАТОРНАЯ РАБОТА №4: Метод bootstrap

Направление подготовки 02.03.01 СЦТ «Сквозные цифровые технологии»,
образовательная программа «Математика и компьютерные науки»,
очная форма обучения

Студент группы Б9123-02.03.01сцт2

_____ Ж. О. Избосарова

Руководитель проекта

_____ К. А. Ганжа

Владивосток

2026

1 Математическое описание метода

Нужно реализовать собственную функцию `bootstrap`, оценить параметры распределений из лабораторной работы 1 и сравнить результаты со встроенной функцией `scipy.stats.bootstrap`.

Во всех вычислениях использовались те же выборки что и в лабораторной работе 1:

- $U(2, 7)$ при $n = 100$ и $n = 1000$;
- $Bernoulli(0.27)$ при $n = 100$ и $n = 1000$;
- $Bin(12, 0.35)$ при $n = 100$ и $n = 1000$;
- $N(4.5, 1.8^2)$ при $n = 100$ и $n = 1000$.

1.1 Метод `bootstrap`

Пусть дана выборка $X_1, X_2, \dots, X_n$ из распределения F , и требуется построить доверительный интервал для параметра θ .

1. Сначала вычисляется точечная оценка $\hat{\theta}_n = T(X_1, \dots, X_n)$. Если эта статистика состоятельна, то

$$\hat{\theta}_n \xrightarrow{P} \theta, \quad n \rightarrow \infty.$$

2. генерируются выборки

$$\hat{F}_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{X_i \leq x\}.$$

По теореме Гливенко–Кантелли

$$\sup_x |\hat{F}_n(x) - F(x)| \xrightarrow{a.s.} 0,$$

3. Для каждой выборки вычисляется новая оценка

$$\theta_i^* = T(X_1^{*(i)}, X_2^{*(i)}, \dots, X_n^{*(i)}).$$

4. После сортировки оценок доверительный-интервал строится: Если γ доверительный уровень, $\alpha = 1 - \gamma$:

$$\left(\theta^*_{(\lfloor \frac{\alpha}{2} k \rfloor)}, \theta^*_{(\lceil (1 - \frac{\alpha}{2}) k \rceil)} \right).$$

В работе использовались параметры

$$\gamma = 0.95, \quad k = 10000.$$

Собственная реализация percentile-bootstrap имеет вид:

```
1 def bootstrap_percentile(sample, estimator, confidence_level
  =0.95,
2                               n_resamples=9999, random_state=0):
3     sample = np.asarray(sample)
4     n = sample.size
5     rng = np.random.default_rng(random_state)
6
7     theta_hat = float(estimator(sample))
8     bootstrap_estimates = np.empty(n_resamples, dtype=float)
9
10    for i in range(n_resamples):
11        bootstrap_sample = rng.choice(sample, size=n, replace
12        =True)
13        bootstrap_estimates[i] = float(estimator(
14        bootstrap_sample))
15
16    bootstrap_estimates.sort()
17    alpha = 1.0 - confidence_level
18    lower_rank = max(1, int(np.floor(alpha / 2.0 *
19    n_resamples)))
20    upper_rank = min(n_resamples, int(np.ceil((1.0 - alpha /
21    2.0) * n_resamples)))
22
23    return {
24        "estimate": theta_hat,
25        "confidence_interval": (
26            float(bootstrap_estimates[lower_rank - 1]),
27            float(bootstrap_estimates[upper_rank - 1]),
```

```

24         ),
25     }

```

Для сравнения использовалась встроенная функция из SciPy:

```

1 result = st.bootstrap(
2     (np.asarray(sample),),
3     estimator,
4     n_resamples=n_resamples,
5     confidence_level=confidence_level,
6     method="percentile",
7     vectorized=False,
8     random_state=random_state,
9 )

```

2 Состоятельные оценки параметров

Обозначим

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i, \quad S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2.$$

По закону больших чисел

$$\bar{X} \xrightarrow{P} EX, \quad S_n^2 \xrightarrow{P} D(X).$$

Далее используется теорема о непрерывном отображении.

2.1 Равномерное распределение $U(a, b)$

Для равномерного распределения

$$EX = \frac{a+b}{2}, \quad D(X) = \frac{(b-a)^2}{12}.$$

Отсюда оценки методом моментов имеют вид

$$\hat{a} = \bar{X} - \sqrt{3S_n^2}, \quad \hat{b} = \bar{X} + \sqrt{3S_n^2}.$$

2.2 Распределение Бернулли $Bernoulli(p)$

В этом случае

$$EX = p,$$

поэтому естественная оценка

$$\hat{p} = \bar{X}$$

2.3 Биномиальное распределение $Bin(m, p)$

Для биномиального распределения

$$EX = mp, \quad D(X) = mp(1 - p).$$

Решая систему уравнений метода моментов, получаем

$$\hat{m} = \frac{\bar{X}^2}{\bar{X} - S_n^2}, \quad \hat{p} = \frac{\bar{X}}{\hat{m}}.$$

Поскольку

$$(\bar{X}, S_n^2) \xrightarrow{P} (mp, mp(1 - p)),$$

имеем $\hat{m} \xrightarrow{P} m$ и $\hat{p} \xrightarrow{P} p$.

2.4 Нормальное распределение $N(\mu, \sigma^2)$

Для нормального распределения использовались оценки

$$\hat{\mu} = \bar{X}, \quad \hat{\sigma} = \sqrt{S_n^2}.$$

3 Результаты

Ниже приведено сравнение точечных оценок и 95%-х доверительных интервалов, построенных с помощью собственной реализации bootstrap и percentile-bootstrap из SciPy.

Таблица 1: Сводное сравнение точечных оценок и 95%-х bootstrap-интервалов

Выборка	Истинные метры	пара- Точечные оценки	Свой bootstrap (95%)	SciPy bootstrap (95%)	Макс. раз-ность
U(2,7), n=100	$a = 2, b = 7$	$a = 1.9756; b = 6.9892$	$a: [1.6438; 2.3668]; b: [6.6102; 7.3090]$	$a: [1.6405; 2.3750]; b: [6.5969; 7.3154]$	0.0132
U(2,7), n=1000	$a = 2, b = 7$	$a = 1.9499; b = 6.9520$	$a: [1.8416; 2.0628]; b: [6.8350; 7.0641]$	$a: [1.8428; 2.0636]; b: [6.8340; 7.0625]$	0.0016
Bernoulli(0.27), n=100	$p = 0.27$	$p = 0.2500$	$p: [0.1700; 0.3400]$	$p: [0.1700; 0.3400]$	0.0000
Bernoulli(0.27), n=1000	$p = 0.27$	$p = 0.2500$	$p: [0.2240; 0.2770]$	$p: [0.2230; 0.2770]$	0.0010
Bin(12,0.35), n=100	$m = 12, p = 0.35$	$m = 10.3509; p = 0.4048$	$m: [7.9789; 15.7990]; p: [0.2656; 0.5350]$	$m: [8.0000; 15.7700]; p: [0.2633; 0.5384]$	0.0290
Bin(12,0.35), n=1000	$m = 12, p = 0.35$	$m = 12.2600; p = 0.3372$	$m: [10.5533; 14.7624]; p: [0.2802; 0.3932]$	$m: [10.4875; 14.7364]; p: [0.2807; 0.3914]$	0.0658
N(4.5,1.8 ²), n=100	$\mu = 4.5, \sigma = 1.8$	$\mu = 4.5379; \sigma = 1.7762$	$\mu: [4.2003; 4.8850]; \sigma: [1.5242; 1.9999]$	$\mu: [4.1812; 4.8863]; \sigma: [1.5298; 1.9955]$	0.0191
N(4.5,1.8 ²), n=1000	$\mu = 4.5, \sigma = 1.8$	$\mu = 4.4620; \sigma = 1.7816$	$\mu: [4.3488; 4.5747]; \sigma: [1.7036; 1.8589]$	$\mu: [4.3518; 4.5727]; \sigma: [1.7020; 1.8565]$	0.0029

Таблица 2: Детальное сравнение bootstrap-интервалов по каждому параметру

Выборка	Параметр	Истинное значение	Точечная оценка	Свой bootstrap	SciPy bootstrap	Макс. раз-ность
U(2,7), n=100	a	2.00	1.9756	[1.6438; 2.3668]	[1.6405; 2.3750]	0.0082
U(2,7), n=100	b	7.00	6.9892	[6.6102; 7.3090]	[6.5969; 7.3154]	0.0132
U(2,7), n=1000	a	2.00	1.9499	[1.8416; 2.0628]	[1.8428; 2.0636]	0.0012
U(2,7), n=1000	b	7.00	6.9520	[6.8350; 7.0641]	[6.8340; 7.0625]	0.0016
Bernoulli(0.27), n=100	p	0.27	0.2500	[0.1700; 0.3400]	[0.1700; 0.3400]	0.0000
Bernoulli(0.27), n=1000	p	0.27	0.2500	[0.2240; 0.2770]	[0.2230; 0.2770]	0.0010
Bin(12,0.35), n=100	m	12.00	10.3509	[7.9789; 15.7990]	[8.0000; 15.7700]	0.0290
Bin(12,0.35), n=100	p	0.35	0.4048	[0.2656; 0.5350]	[0.2633; 0.5384]	0.0034
Bin(12,0.35), n=1000	m	12.00	12.2600	[10.5533; 14.7624]	[10.4875; 14.7364]	0.0658
Bin(12,0.35), n=1000	p	0.35	0.3372	[0.2802; 0.3932]	[0.2807; 0.3914]	0.0018
N(4.5,1.8 ²), n=100	μ	4.50	4.5379	[4.2003; 4.8850]	[4.1812; 4.8863]	0.0191
N(4.5,1.8 ²), n=100	σ	1.80	1.7762	[1.5242; 1.9999]	[1.5298; 1.9955]	0.0056
N(4.5,1.8 ²), n=1000	μ	4.50	4.4620	[4.3488; 4.5747]	[4.3518; 4.5727]	0.0029
N(4.5,1.8 ²), n=1000	σ	1.80	1.7816	[1.7036; 1.8589]	[1.7020; 1.8565]	0.0024

4 Вывод

При переходе от $n = 100$ к $n = 1000$ доверительные интервалы становятся уже, что согласуется с ожидаемым уменьшением разброса оценок при росте объёма выборки.

Собственная реализация bootstrap и bootstrap из SciPy дают близкие результаты для рассмотренных распределений.

Наибольшее расхождение было для выборки $Bin(12, 0.35)$ при $n = 1000$

и параметра m_b максимальная разность между границами интервалов составила 0.0658.

5 Полный код программы

```
1 # Exported from lab4.ipynb for inclusion in the LaTeX report.
2
3 import math
4 import numpy as np
5 import pandas as pd
6 import scipy.stats as st
7
8 RANDOM_STATE = 16
9 BOOTSTRAP_RANDOM_STATE = 2026
10 CONFIDENCE_LEVEL = 0.95
11 N_RESAMPLES = 10_000
12
13 pd.set_option("display.max_colwidth", None)
14
15
16 def build_samples(random_state=RANDOM_STATE):
17     return {
18         "uniform_100": st.uniform.rvs(loc=2, scale=5, size
19         =100, random_state=random_state),
20         "uniform_1000": st.uniform.rvs(loc=2, scale=5, size
21         =1000, random_state=random_state),
22         "bernoulli_100": st.bernoulli.rvs(p=0.27, size=100,
23         random_state=random_state),
24         "bernoulli_1000": st.bernoulli.rvs(p=0.27, size=1000,
25         random_state=random_state),
26         "binom_100": st.binom.rvs(n=12, p=0.35, size=100,
27         random_state=random_state),
28         "binom_1000": st.binom.rvs(n=12, p=0.35, size=1000,
29         random_state=random_state),
30         "norm_100": st.norm.rvs(loc=4.5, scale=1.8, size=100,
31         random_state=random_state),
32         "norm_1000": st.norm.rvs(loc=4.5, scale=1.8, size
33         =1000, random_state=random_state),
34     }
```

```

27
28
29 samples = build_samples()
30
31
32 def sample_mean(sample):
33     sample = np.asarray(sample, dtype=float)
34     return float(np.mean(sample))
35
36
37 def sample_var(sample):
38     sample = np.asarray(sample, dtype=float)
39     centered = sample - np.mean(sample)
40     return float(np.mean(centered ** 2))
41
42
43 def estimate_uniform_params(sample):
44     mu = sample_mean(sample)
45     var = sample_var(sample)
46     half_width = math.sqrt(max(3.0 * var, 0.0))
47     return mu - half_width, mu + half_width
48
49
50 def estimate_uniform_a(sample):
51     return estimate_uniform_params(sample)[0]
52
53
54 def estimate_uniform_b(sample):
55     return estimate_uniform_params(sample)[1]
56
57
58 def estimate_bernoulli_p(sample):
59     return sample_mean(sample)
60
61
62 def estimate_binom_params(sample):
63     x = np.asarray(sample, dtype=float)
64     mu = sample_mean(x)
65     var = sample_var(x)
66     eps = 1e-12

```

```

67
68     if mu <= eps:
69         return 1.0, 0.0
70
71     denominator = mu - var
72     if denominator <= eps:
73         m_hat = max(float(np.max(x)), 1.0)
74     else:
75         m_hat = max(mu * mu / denominator, float(np.max(x)),
76 1.0)
77
78     p_hat = min(max(mu / m_hat, eps), 1 - eps)
79     return float(m_hat), float(p_hat)
80
81 def estimate_binom_m(sample):
82     return estimate_binom_params(sample)[0]
83
84
85 def estimate_binom_p(sample):
86     return estimate_binom_params(sample)[1]
87
88
89 def estimate_normal_mu(sample):
90     return sample_mean(sample)
91
92
93 def estimate_normal_sigma(sample):
94     return math.sqrt(max(sample_var(sample), 0.0))
95
96
97 def bootstrap_percentile(
98     sample,
99     estimator,
100     confidence_level=0.95,
101     n_resamples=9999,
102     random_state=0,
103 ):
104     sample = np.asarray(sample)
105     n = sample.size

```

```

106     rng = np.random.default_rng(random_state)
107
108     theta_hat = float(estimator(sample))
109     bootstrap_estimates = np.empty(n_resamples, dtype=float)
110
111     for i in range(n_resamples):
112         bootstrap_sample = rng.choice(sample, size=n, replace
113 =True)
114         bootstrap_estimates[i] = float(estimator(
115 bootstrap_sample))
116
117     bootstrap_estimates.sort()
118     alpha = 1.0 - confidence_level
119     lower_rank = max(1, int(np.floor(alpha / 2.0 *
120 n_resamples)))
121     upper_rank = min(n_resamples, int(np.ceil((1.0 - alpha /
122 2.0) * n_resamples)))
123
124     return {
125         "estimate": theta_hat,
126         "confidence_interval": (
127             float(bootstrap_estimates[lower_rank - 1]),
128             float(bootstrap_estimates[upper_rank - 1]),
129         ),
130         "bootstrap_distribution": bootstrap_estimates,
131     }
132
133 def scipy_percentile_bootstrap(
134     sample,
135     estimator,
136     confidence_level=0.95,
137     n_resamples=9999,
138     random_state=0,
139 ):
140     result = st.bootstrap(
141         (np.asarray(sample),),
142         estimator,
143         n_resamples=n_resamples,
144         confidence_level=confidence_level,

```

```

142     method="percentile",
143     vectorized=False,
144     random_state=random_state,
145 )
146     return float(result.confidence_interval.low), float(
147         result.confidence_interval.high)
148
149 EXPERIMENTS = [
150     {
151         "sample_key": "uniform_100",
152         "label": "U(2,7), n=100",
153         "true_parameters": {"a": 2.0, "b": 7.0},
154         "estimators": {"a": estimate_uniform_a, "b":
155             estimate_uniform_b},
156     },
157     {
158         "sample_key": "uniform_1000",
159         "label": "U(2,7), n=1000",
160         "true_parameters": {"a": 2.0, "b": 7.0},
161         "estimators": {"a": estimate_uniform_a, "b":
162             estimate_uniform_b},
163     },
164     {
165         "sample_key": "bernoulli_100",
166         "label": "Bernoulli(0.27), n=100",
167         "true_parameters": {"p": 0.27},
168         "estimators": {"p": estimate_bernoulli_p},
169     },
170     {
171         "sample_key": "bernoulli_1000",
172         "label": "Bernoulli(0.27), n=1000",
173         "true_parameters": {"p": 0.27},
174         "estimators": {"p": estimate_bernoulli_p},
175     },
176     {
177         "sample_key": "binom_100",
178         "label": "Bin(12,0.35), n=100",
179         "true_parameters": {"m": 12.0, "p": 0.35},

```

```

178         "estimators": {"m": estimate_binom_m, "p":
estimate_binom_p},
179     },
180     {
181         "sample_key": "binom_1000",
182         "label": "Bin(12,0.35), n=1000",
183         "true_parameters": {"m": 12.0, "p": 0.35},
184         "estimators": {"m": estimate_binom_m, "p":
estimate_binom_p},
185     },
186     {
187         "sample_key": "norm_100",
188         "label": "N(4.5,1.8^2), n=100",
189         "true_parameters": {"mu": 4.5, "sigma": 1.8},
190         "estimators": {"mu": estimate_normal_mu, "sigma":
estimate_normal_sigma},
191     },
192     {
193         "sample_key": "norm_1000",
194         "label": "N(4.5,1.8^2), n=1000",
195         "true_parameters": {"mu": 4.5, "sigma": 1.8},
196         "estimators": {"mu": estimate_normal_mu, "sigma":
estimate_normal_sigma},
197     },
198 ]
199
200
201 def format_interval(interval):
202     return f"[{interval[0]:.4f}; {interval[1]:.4f}]"
203
204
205 summary_rows = []
206 detail_rows = []
207
208 for experiment in EXPERIMENTS:
209     sample = samples[experiment["sample_key"]]
210     estimate_chunks = []
211     custom_chunks = []
212     scipy_chunks = []
213     max_difference = 0.0

```

```

214
215     for offset, (parameter_name, estimator) in enumerate(
experiment["estimators"].items()):
216         custom_result = bootstrap_percentile(
217             sample,
218             estimator,
219             confidence_level=CONFIDENCE_LEVEL,
220             n_resamples=N_RESAMPLES,
221             random_state=BOOTSTRAP_RANDOM_STATE + offset,
222         )
223         scipy_interval = scipy_percentile_bootstrap(
224             sample,
225             estimator,
226             confidence_level=CONFIDENCE_LEVEL,
227             n_resamples=N_RESAMPLES,
228             random_state=BOOTSTRAP_RANDOM_STATE + offset,
229         )
230
231         custom_interval = custom_result["confidence_interval"
]
232         difference = max(
233             abs(custom_interval[0] - scipy_interval[0]),
234             abs(custom_interval[1] - scipy_interval[1]),
235         )
236         max_difference = max(max_difference, difference)
237
238         estimate_chunks.append(f"{parameter_name}={
custom_result['estimate']:.4f}")
239         custom_chunks.append(f"{parameter_name}: {
format_interval(custom_interval)}")
240         scipy_chunks.append(f"{parameter_name}: {
format_interval(scipy_interval)}")
241
242         detail_rows.append(
243             {
244                 "sample": experiment["label"],
245                 "parameter": parameter_name,
246                 "true_value": experiment["true_parameters"][
parameter_name],
247                 "point_estimate": custom_result["estimate"],

```

```

248         "custom_bootstrap": format_interval(
custom_interval),
249         "scipy_bootstrap": format_interval(
scipy_interval),
250         "max_abs_diff": difference,
251     }
252 )
253
254 summary_rows.append(
255     {
256         "sample": experiment["label"],
257         "true_parameters": ", ".join(
258             f"{name}={value:g}" for name, value in
experiment["true_parameters"].items()
259         ),
260         "point_estimates": "; ".join(estimate_chunks),
261         "custom_bootstrap_95": "; ".join(custom_chunks),
262         "scipy_bootstrap_95": "; ".join(scipy_chunks),
263         "max_abs_diff": max_difference,
264     }
265 )
266
267 summary_df = pd.DataFrame(summary_rows)
268 detail_df = pd.DataFrame(detail_rows)
269 detail_display_df = detail_df.copy()
270
271 for column in ["true_value", "point_estimate", "max_abs_diff"
272 ]:
273     detail_display_df[column] = detail_display_df[column].
astype(float).round(4)
274
275 summary_df["max_abs_diff"] = summary_df["max_abs_diff"].
astype(float).round(4)
276
277 print("Summary table")
278 print(summary_df.to_string(index=False))
279 print()
280 print("Detailed table")
281 print(detail_display_df.to_string(index=False))

```

```

282 print()
283
284 largest_gap_row = detail_display_df.loc[detail_display_df["
    max_abs_diff"].idxmax()]
285 print(
286     "Conclusion: intervals become narrower when n grows from
    100 to 1000. "
287     "The custom percentile bootstrap and scipy.stats.
    bootstrap(..., method='percentile') "
288     "give close results. "
289     f"The largest gap is for sample {largest_gap_row['sample
    ']} and parameter "
290     f"{largest_gap_row['parameter']}; the maximum absolute
    difference is "
291     f"{largest_gap_row['max_abs_diff']:.4f}."
292 )
293
294 summary_df

```

Листинг 1: Код программы для лабораторной работы 4